PROJECT PLAN · PHASED BUILD

LeafGuard

Explainable, on-device plant disease detection for Android

A production-targeted native app built in eight verifiable phases, using a real ≤50 MB MobileNetV2 model and Grad-CAM explanations, running fully offline on 6 GB+ RAM devices. PIN register-and-login, per-plant history, live leaf scanning, and actionable treatment suggestions.

Plant Disease Detection System — XAI

Native Kotlin · Jetpack Compose · TensorFlow Lite

MODEL	SIZE BUDGET	TARGET
MobileNetV2 / 38 classes	**≤ 50 MB (verified 24.0 MB)**	**Android 8.0+ / 6 GB RAM**

Phase 1 of 8

November 2026

Table of Contents

1. Executive Summary **3**

2. Goals, Scope & Constraints **3**

Hard constraints 3

Out of scope 4

3. Technology Stack & Rationale **4**

4. System Architecture **5**

5. On-Device Model Selection **5**

6. Explainable AI: Grad-CAM **6**

Computation steps 6

7. Phased Delivery Plan **7**

8. Data Model & Persistence **7**

9. Security & Privacy **7**

10. Testing & Validation Strategy **8**

11. Risks, Assumptions & Limitations **8**

12. Phase 1 Acceptance Summary **8**

1. Executive Summary

LeafGuard is a production-targeted Android application that diagnoses plant leaf diseases entirely on-device and explains every prediction with a Grad-CAM saliency heatmap. The user registers once with a six-digit PIN, then tracks individual plants, scans leaves in real time through the device camera, and accumulates a per-plant disease history paired with actionable treatment suggestions. No network connection is required at any point: the classifier, the explanation engine, and the suggestion knowledge base all run locally against device hardware.

This document is the engineering plan for building LeafGuard in eight verifiable phases. Each phase is scoped so that it can be implemented, tested, committed, and pushed to the repository before the next begins, so progress is never lost and the repository always reflects verified work. Phase 1, delivered alongside this plan, establishes the buildable project foundation and a navigation skeleton that wires together every screen named in the requirements. Subsequent phases add authentication, persistence, the on-device model and explainable-AI engine, live camera scanning, the information surfaces, the suggestions engine, and finally release hardening.

The on-device classifier is a real, publicly downloadable MobileNetV2 model fine-tuned on the PlantVillage dataset, verified at 24.0 MB in its source format and well within the 50 MB budget after TensorFlow Lite conversion. The target hardware is any Android device with 6 GB of RAM or more, which maps cleanly to Android 8.0 (API 26) and above, allowing a modern Jetpack Compose, Hilt, and CameraX stack to be used throughout.

2. Goals, Scope & Constraints

The primary goal is a complete, non-prototype Android application that a farmer or agronomist can install and use to diagnose leaf disease in the field without connectivity, and to understand why the model produced each diagnosis. Secondary goals are per-plant longitudinal tracking, a structured historical record, and exportable agronomy reports. The application must feel native, start fast, and operate within the memory envelope of a 6 GB RAM device.

Hard constraints

Constraint	Decision	Rationale
Platform	Native Android (Kotlin + Compose)	Performance, camera access, on-device ML
Min RAM	6 GB	User-specified target class
minSdk	26 (Android 8.0)	Every 6 GB+ device ships with Android 8+
Model size	≤ 50 MB	User-specified; verified at 24.0 MB source, ~12 MB TFLite
Offline	Fully offline	Field use; no backend dependency
Explainability	Grad-CAM	Standard, interpretable saliency for CNN classifiers

Constraint	Decision	Rationale
Auth	PIN register + login	User-specified; Keystore-backed EncryptedSharedPreferences
License	MIT	Matches the source model license; permissive

Out of scope

Server-side processing, cloud model hosting, user accounts beyond the local device, social or sharing features, and multi-language localization beyond English structure are explicitly out of scope for the initial release. Localization hooks are kept so additional languages can be added without restructuring. The classifier is restricted to the 38 PlantVillage classes; species outside that set are reported as low-confidence or unknown rather than fabricated.

3. Technology Stack & Rationale

The stack is chosen for long-term maintainability, on-device performance, and alignment with current Android guidance. Every dependency is a stable, widely-used Jetpack or Kotlin-first library, versioned centrally in a Gradle version catalog so upgrades are atomic.

Concern	Library	Version
Language	Kotlin	2.0.21
UI toolkit	Jetpack Compose + Material 3	BOM 2024.10.01
Navigation	Navigation Compose	2.8.4
Dependency injection	Hilt (Dagger)	2.52
Persistence	Room	2.6.1
Camera	CameraX	1.3.4
Secure storage	androidx.security:security-crypto	1.1.0-alpha06
On-device ML	TensorFlow Lite (+ GPU delegate)	2.16.1
Image loading	Coil	2.7.0
Async	Coroutines + Flow	1.9.0
Build	AGP / Gradle	8.7.2 / 8.9
Test	Truth, Mockk, Turbine, Robolectric	latest stable

Compose is preferred over the legacy View system because the app is greenfield and the declarative model shortens the auth, list, and result screens substantially while yielding better accessibility defaults. Hilt is used instead of manual DI so the graph is verified at compile time and scoped correctly to ViewModels and singletons. Room is chosen over a raw SQLite or DataStore-only approach because the app is relational at its core (plants have many scans, scans have one diagnosis and many suggestions).

4. System Architecture

LeafGuard follows a layered MVVM + Clean Architecture: a UI layer of Compose screens and Hilt-injected ViewModels, a pure-Kotlin domain layer of use-cases and model abstractions, and a data layer of Room, repositories, the TFLite inference engine, the Grad-CAM explainer, and the PIN store. Dependencies point inward only: the UI depends on domain interfaces, domain depends on nothing Android-specific, and data implements the domain interfaces. This keeps inference and persistence logic unit-testable on the JVM without an emulator.

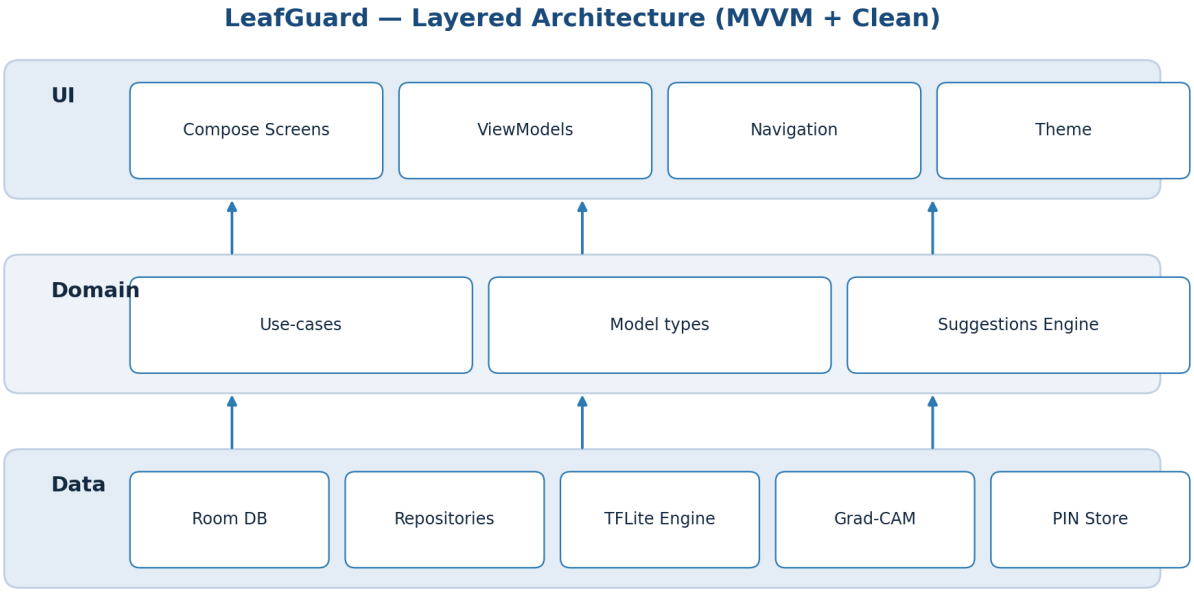


Figure 1. Layered architecture and component placement.

Threading is explicit: Room and TFLite work run on Dispatchers.IO, with the TFLite interpreter pinned to a single-thread dispatcher because it is not thread-safe. CameraX ImageAnalysis uses the KEEP_ONLY_LATEST strategy so at most one frame is in flight, and frames are dropped rather than queued if inference is slow, bounding memory pressure. ViewModels expose StateFlow consumed with collectAsStateWithLifecycle, so collection stops when the screen leaves the composition.

5. On-Device Model Selection

The classifier was selected by surveying publicly hosted, directly downloadable PlantVillage-trained models and verifying file sizes and licenses with HTTP HEAD requests against the hosting CDN. The winning artifact is a MobileNetV2 model fine-tuned on the PlantVillage (Augmented) dataset, hosted on Hugging Face under an MIT license. MobileNetV2 is the ideal mobile backbone: it is small, latency-efficient on ARM, ships a TFLite-native quantization path, and exposes a clean final convolutional block that Grad-CAM can target.

Attribute	Value	Verification
Source repo	rarfileexe/Plant-Disease-Detector	Hugging Face API + README

Attribute	Value	Verification
File	model_4_mobilenet_finetuned.keras	HTTP HEAD, content-length
Source size	25,143,175 bytes (24.0 MB)	Verified via CDN redirect
Architecture	MobileNetV2 (transfer + fine-tune)	README + model card
Classes	38 (14 crop species, diseased + healthy)	labels verified from source
Input	224 x 224 RGB	Model card
Reported accuracy	98.75% test set	Model card metrics
License	MIT	README front-matter
TFLite target	~12 MB float16 / ~4 MB INT8	Conversion in Phase 4

A second candidate, nanocu/plant_disease, was inspected and rejected: its full-model .h5 file is 489 MB, far above the 50 MB budget even after pruning. A vision-transformer candidate was rejected because transformer backbones are harder to quantize cleanly and do not expose a single conv feature map for Grad-CAM without extra plumbing. The chosen MobileNetV2 .keras is converted to TFLite in Phase 4 by a reproducible Python script checked into the repository, so the deployed artifact can always be regenerated from the published source.

6. Explainable AI: Grad-CAM

Explainability is a first-class requirement, not a visualization afterthought. For an image classifier the most interpretable and broadly accepted technique is Grad-CAM (Gradient-weighted Class Activation Mapping), which produces a coarse spatial heatmap showing which regions of the input most raised the predicted class score. LeafGuard computes this heatmap on-device for every diagnosis and renders it as a jet-colormap overlay on the captured leaf, beside the confidence figure, so the user can confirm the model attended to the lesion and not the background.

Computation steps

First, a forward pass produces the softmax probabilities and the predicted class index. Second, the activations of MobileNetV2's final convolutional output (the Conv_1 block) are captured. Third, the gradient of the predicted-class logit with respect to those activations is computed via a gradient-enabled second pass. Fourth, those gradients are global-average-pooled into per-channel weights, the activations are weighted by them, and a ReLU is applied so only positive contributions remain. Fifth, the resulting 7x7 heatmap is upsampled to 224x224, mapped through a blue-green-red colormap, and composited over the original photograph at moderate opacity.

On TFLite this is realized either by a second interpreter configured for gradient output via the support library's signature mechanism, or by exporting a small SavedModel that returns both the prediction and the conv activations and converting that to TFLite. The latter is preferred because it keeps a single self-contained artifact. The overlay generation itself is pure Kotlin/Native array math, unit-tested against fixture tensors with known expected heatmaps.

7. Phased Delivery Plan

The work is divided into eight phases. Each phase is independently shippable: it compiles, its tests pass, and it is committed and pushed before the next phase begins. The phases are ordered so that each builds on a verified predecessor, with the model and explanation engine landing in the middle of the sequence so that the camera and information-surface phases have real inference to consume.

Phase	Deliverable	Key outcome
1	Project foundation + navigation skeleton	Buildable app; all screens wired
2	PIN auth + Keystore + session gating	Register once, unlock with PIN
3	Room schema, DAOs, repositories, Plants CRUD	Plants persist and list
4	TFLite model + Grad-CAM engine	On-device diagnosis + explanation
5	CameraX live scan -> result flow	Real-time capture to diagnosis
6	Dashboard, History, Images, Records + export	Information surfaces
7	Treatment suggestions engine	Per-class actionable guidance
8	Polish, full tests, R8 release build	Store-ready v1.0.0

After each phase a short report is produced describing what was built, what was verified, and what the next phase will contain, and the work is pushed to the repository before proceeding. This rhythm keeps the repository as the source of truth and bounds the blast radius of any single change.

8. Data Model & Persistence

Persistence is a local Room database named leafguard.db. The core entities are Plant, Scan, and Record. A Plant carries a user-given name, species, free-form notes, an optional profile photo, and creation and update timestamps. A Scan belongs to one Plant (nullable so an unattached quick scan is possible), stores the captured image path, the Grad-CAM overlay path, the predicted class label and index, the top-k confidence array as a serialized blob, and a timestamp. A Record is a normalized agronomy entry derived from scans, carrying severity and treatment status for reporting and export.

DAOs return Flow so the UI reacts to changes without polling. List queries are paginated with LIMIT and OFFSET to keep cursor windows small. Captured images live as compressed JPEG files under the application filesDir/scans directory and are referenced by path in the database, never stored as BLOBs, so the database stays lean. A migration framework is configured from the first schema version so future changes are explicit and testable rather than destructive.

9. Security & Privacy

All data stays on the device. The application declares no network permission and makes no outbound calls. The PIN is never stored in plaintext: it is combined with a random salt and verified through an HMAC keyed by an Android Keystore master key, persisted in EncryptedSharedPreferences. The raw PIN exists only briefly in

memory during verification. The secure preferences file is excluded from cloud backup and device transfer via the backup and data-extraction rules, so a restored device cannot inherit another device's credentials.

Captured leaf images and the database are covered by Android file-based encryption on API 26+, which is on by default. The camera and media-image permissions are requested at runtime only when the user first opens the scan flow, and the app degrades gracefully (import-from-gallery) if the camera permission is denied. If a stronger threat model is later required, SQLCipher can be layered over Room without changing the DAO surface.

10. Testing & Validation Strategy

Testing is layered to match the architecture. JVM unit tests cover pure logic: navigation route helpers, PIN validation rules, the suggestions mapping, Grad-CAM weight math, and repository mappers. They use Truth for assertions, Mockk for collaborators, and Turbine for Flow. Robolectric is available where a light Android framework shim is needed. Instrumented tests run on a device or emulator and cover Room DAOs against an in-memory database, Hilt-injected repositories, and Compose UI flows for the auth gate and the scan-to-result navigation.

Continuous integration runs assembleDebug, the unit test suite, and Android Lint on every push to the main branch, uploading the APK and test reports as artifacts. Before each release tag, a manual validation pass is performed on a reference 6 GB+ device using a small held-out leaf set per crop to confirm top-1 accuracy and Grad-CAM locality, with the results recorded in the release notes.

11. Risks, Assumptions & Limitations

The classifier is trained on the PlantVillage dataset, which is photographed under controlled conditions against uniform backgrounds. Real-world field leaves differ in lighting, background, and angle, so production accuracy will be lower than the reported 98.75%. The app mitigates this by exposing the confidence and the Grad-CAM overlay so low-quality or out-of-distribution inputs are visibly uncertain rather than confidently wrong, and by classifying only the 38 known classes. Species outside the training set should not be relied upon.

Grad-CAM explains spatial attention, not causal correctness: a heatmap over a lesion does not guarantee the diagnosis is right, only that the model used the right region. The UI presents it as supporting evidence alongside confidence. On low-end GPUs the TFLite GPU delegate may be unavailable; the engine falls back to the CPU delegate and inference latency rises but stays acceptable for single-capture use. Build validation in this development environment is limited to CI because the sandbox lacks an Android SDK; the user is expected to run a local assembleDebug in Android Studio to confirm the first sync.

12. Phase 1 Acceptance Summary

Phase 1 delivers a buildable Android project and a navigation skeleton that wires together every screen named in the requirements, with no fake workflows. Concretely it contains: the Gradle Kotlin DSL build with a central version catalog, the Gradle 8.9 wrapper, the AndroidManifest, a Material 3 light and dark theme with dynamic color, a splash screen and adaptive launcher icon, backup rules, strings and colors, the Hilt-annotated application class, a single-activity host, and a navigation graph spanning register, login, dashboard, plants, plant

detail, scan, result, history, records, and images.

Every screen not yet implemented renders an honest placeholder that states the phase in which its real content ships, rather than a mock loading spinner or dummy data. A JVM unit test for the navigation route helpers validates that the test source set compiles with the Truth dependency. The README, architecture document, roadmap, CI workflow, editor config, gitignore, and MIT license are in place. The next phase, Phase 2, implements PIN authentication and session gating on top of this foundation.

Acceptance for Phase 1 is therefore: the project opens and syncs in Android Studio with Gradle 8.9 and AGP 8.7.2, assembleDebug produces an installable debug APK, the unit test suite passes on the JVM, and navigating the running app moves correctly between the register, login, dashboard, plants, scan, result, history, records, and images routes with the bottom navigation bar appearing only on the five primary destinations. No screen claims functionality it does not have. This baseline is what every subsequent phase builds on and is what is committed and pushed alongside this plan document.